LAB SESSION NO. 2 Part-1

PROLOG-KEYWORDS AND ARITHMATIC OPERATIONS

How to add a fact with a program?

It is possible to add new facts with the instruction assert(fact1) which will add the fact called fact1. The predicates added with this command are considered like any other in the source of the program.

MORE ON PATTERN MATCHING (UNIFY).

Prolog matches expressions in a purely structural way, with no evaluation of expressions, so:

```
?- 1+2 = 3

no
```

[Note: In prolog ``=" means ``matches with".]

Similarly the following won't match, as the two expressions have different structures:

```
?- X + 2 = 3 * Y.

no
```

However, the following match as they have the same structure:

```
?- X = v1,(X = v1; X = v3).

X = v1 ;

false

?- X = Val,(X = v1; X = v3).

Val = X, X = v1 ;

Val = X, X = v3
```

?- X = v3,(X = v1; X = v3).

X = v3

?- X = v3,(X = v1; X = v4).

no

?- X+Y = 1+2.

X = 1

Y = 2

?- X*Y = 1+2.

no

?- 1+Y = X + 3.

X = 1

Y = 3

?- X+3=3+Y.

X = Y, Y = 3

Note that there can be variables in both sides of a matched expression, as in the second example above.

For non-arithmetic examples, we have:

?- lectures(X, ai) = lectures(ali, Y).

X = ali

Y = ai

?- book(title, author(Y)) = book(Z, author(bratco)).

Y = bratco ,

Z = title

?- book(title(X), author(Y)) = book(Z, author(bratco)).

Y=bratco,

Z=tittle(X)

?- book(title(Z), author(Y)) = book(X, author(bratco)).

Y = bratco ,

X = title(Z)

(Actually, in prolog you'd get something looking a bit more complex in response to the above query, with maybe Z = title(_) and X = _, as prolog creates its own internal variables. This is equivalent to the bindings given above.)

Of course, prolog will normally be doing lots of matches in a row, as it tries to prove different subgoals in a rule. It then needs to use the variable bindings obtained in the matches so far when it does the next match. So we might have:

?- X+Y = 1+5, X=Y.

no

| ?- X+Y = 1+1, X=Y.

X = Y, Y = 1

?- X+Y = 1+5, X=Y.

no

?- X+Y = 1*1, X=Y.

no

[**Note:** We can ask several queries in a row, with a comma in between them, just like in the body of a Prolog rule.]

X and Y get instantiated (bound) to 1 and 5 respectively in the first match (X+Y = 1+5), so the second match is effectively 1=5, which fails. However, the following will succeed:

?- X+Y = 1+5, Z = X.

X=Z, Z = 1,

Y = 5

?- book(author(X)) = book(author(bratco)), famous(X) = famous(bratco).

X = bratco

?- book(author(X)) = book(author(bratco)), famous(Y) = famous(bratco).

X = Y, Y = bratco

?- book(author(X)) = book(author(bratco)), famous(X) = famous(bratco1).

no %(this result given by “,”/and operator because both were not matched).

?- book(author(X)) = book(author(bratco1)), famous(X) = famous(bratco1).

X = bratco1

?- X=Y, Y = alison.

X = Y, Y= alison

Note that if two uninstantiated (unbound) variables are matched (and therefore instantiated to each other) then as soon as one becomes instantiated to some term then the other automatically becomes instantiated to that term.

<u>OUTPUT Keywords: write, nl & display</u>

<u>write</u> takes any prolog term as its argument and displays that term on the screen. Example:

?- write('Assalamualaikum'), write('Warahmatullah').

AssalamualaikumWarahmatullah
yes

?- write('Assalamualaikum'), nl, write('Warahmatullah').

Assalamualaikum

Warahmatullah
yes

```
?- write('Assalamualaikum'),tab(5), write('Warahmatullah').

Assalamualaikum     Warahmatullahyes

?- write('Assalamualaikum'), tab(1), write('Warahmatullah').

Assalamualaikum Warahmatullahyes

?- mother(X,mariam), write('The mother of Mariam is '), write(X). /* add fact first then run this*/

The mother of Mariam is amna
X = amna

?- mother(X,mariam), write('The mother of Mariam is '), tab(1),write(X).

The mother of Mariam is  amna
X = amna

?- write(X).   %assume X in uninstantiated

_27590X

?-writeq('Assalamualaikum'). % it will print everything with single quote.

'Assalamualaikum'yes

| ?- writeq("Assalamualaikum"). % it will print everything with double quote
```

```
"Assalamualaikum"
true

| ?- writeq("#,%").

"#,%"
true
```

<u>Note</u>: display keyword puts all operators in front of their arguments even if they were originally written in other positions (it will change the infix notations into prefix). Useful for investigating the internal representation of Prolog terms.

```
?- display(2+2).

+(2,2)yes

?- display(2+2+9).

+(+(2,2),9)yes
```

?- display(2*9+5).

+(*(2,9),5)yes

?- display(2+2*9).

+(2,*(2,9))yes

| ?- display(2-2+9).

+(-(2,2),9)yes

?- display(2+2-9).

-(+(2,2),9)yes

?- display(2/2+9).

+(/(2,2),9)yes

?- display(2/2*9).

*(/(2,2),9)yes

?- display(2*2/9).

/(*(2,2),9)yes

?- display(2+2*9).

+(2,*(2,9))yes

?- display('hello there').% By using single quote you can print it as you do it using write().

‘hello there’
true

?- display("hello there").

“hello there”
true

?- name(123, X). %It will shoe you the ASCII code of 123 into X variable
X = [49, 50, 51]

4.1 Arithmetic in Prolog

Prolog provides a number of basic arithmetic tools for manipulating integers (that is, numbers of the form ...-3, -2, -1, 0, 1, 2, 3, 4...). Most Prolog implementation also provide tools for handling real numbers (or floating point numbers) such as 1.53 or 6.35×10^5, but we're not going to discuss these, for they are not particularly useful for the symbolic processing tasks discussed in this course. Integers, on the other hand, are useful for various tasks (such as finding the length of a list), so it is important to understand how to work with them. We'll start by looking at how Prolog handles the four basic operations of addition, multiplication, subtraction, and division.

Arithmetic Examples	**Prolog Notations**
6+2=8	8 is 6+2.
6*2=12	12 is 6*2.
6-2=4	4 is 6-2.
6-8=-2	-2 is 6-8.
6%2=3	3 is 6/2.
7%2=3.5	3.5 is 7/2.
1 is the remainder when 7 is divided by 2	1 is mod(7,2).

(Note that as we are working with integers, sometime division gives us back an integer answer. Thus 7\div 2 gives 3 as an answer, leaving a reminder of 1, or 3.5.)

Posing the following queries yields the following responses:

```
?- 8 is 6+2.
yes
?- 12 is 6*2.
yes
?- -2 is 6-8.
yes
```

```
?- 3 is 6/2.
yes
?- 3.5 is 7/2.
yes
?- 7/2 is 3.5.
false
?- 1 is mod(7,2). % if you change the position of operands then result will not come same
true
```

More importantly, we can work out the answers to arithmetic questions by using variables. For example:

```
?- X is 6+2.
X = 8

?- X is 6*2.
X = 12

?- X is 7/2.
X = 3.5

?- R is mod(7,2).
R = 1
```

Moreover, we can use arithmetic operations when we define predicates. Here's a simple example. Let's define a predicate "*add_3_and_double2"* whose arguments are both integers. This predicate takes its first argument, adds three to it, doubles the result, and returns the number obtained as the second argument. We define this predicate in database/knowledge-base first as follows:

The following will be example of writing "Rule" in prolog by using ":-" (colon, dash)

Rule example 1:

add_3_then_double_it(X,Y) **:-** Y is (X+3)*2.

Compile the code and indeed, this works:

```
?- add_3_then_double_it(1,X).
 X = 8

?- add_3_then_double_it(2,X).
 X = 10
```

Rule example 2:

add_3_then_double_it(X,Y) :- Y is X+3*2. % Because of operator precedence

Compile the code and indeed, this works:

?- add_3_then_double_it(1,X).

X = 7

?- add_3_then_double_it(2,X).

X = 8

One other thing. Prolog understands the usual conventions we use for disambiguating arithmetical expressions. For example, when we write 3+2*4 we mean 3+(2*4) and not (3+2)*4, and Prolog knows this convention:

?- X is 3+2*4.
X = 11

4.2 A closer look

That's the basics, but we need to know more. The most important to grasp is this: +, *, -, div and mod do not carry out any arithmetic. In fact, expressions such as 3+2, 3-2 and 3*2 are simply terms. The functors of these terms are +, - and * respectively, and the arguments are 3 and 2. Apart from the fact that the functors go between their arguments (instead of in front of them) these are ordinary Prolog terms, and unless we do something special, Prolog will not actually do any arithmetic. In particular, if we pose the query

?- X = 3+2.

we don't get back the answer X=5. Instead we get back

X = 3+2

yes

That is, Prolog has simply bound the variable X to the complex term 3+2. It has not carried out any arithmetic. It has simply done what it usually does: performed unification. Similarly, if we pose the query

?- 3+2*5 = X.

we get the response

X = 3+2*5

yes

Again, Prolog has simply bound the variable X to the complex term 3+2*5. It did not evaluate this expression to 13. To force Prolog to actually evaluate arithmetic expressions we have to use

is keyword just as we did in our earlier examples. In fact, “is” does something very special: it sends a signal to Prolog that says `Hey! Don't treat this expression as an ordinary complex term! Call up your inbuilt arithmetic capabilities and carry out the calculations!'

In short, “**is**” forces Prolog to act in an unusual way. Normally Prolog is quite happy just unifying variables to structures: that is its job, after all. Arithmetic is something extra that has been bolted on to the basic Prolog engine because it is useful. Unsurprisingly, there are some restrictions on this extra ability, and we need to know what they are.

For a start, the arithmetic expressions to be evaluated must be on the right hand side of is. In our earlier examples we carefully posed the query

```
?- X is 6+2.
X = 8
```

which is the right way to do it. If instead we had asked

```
6+2 is X.
```

We would have got an error message saying Type error, or something similar.

Moreover, although we are free to use variables on the right hand side of is, when we actually carry out evaluation, *the variable must already have been instantiated to an integer*. If the variable is uninstantiated, or if it is instantiated to something other than an integer, we will get some sort of instantiation/type_error message. And this makes perfect sense. Arithmetic *isn't* performed using Prolog usual unification and knowledge base search mechanisms: it's done by calling up a special `black box' which knows about integer arithmetic. If we hand the black box the wrong kind of data, naturally it’s going to complain.

Here's an example. Recall our `add 3 and double it' predicate.

```
add_3_and_double(X,Y) :- Y is (X+3)*2.
```

When we described this predicate, we carefully said that it added 3 to its first argument, doubled the result, and returned the answer in its second argument. For example, add_3_and_double(3,X) returns X = 12. We didn't say anything about using this predicate in the reverse direction. For example, we might hope that posing the query:

```
add_3_and_double(X,12).
```

Would return the answer X=3. But it doesn't! Instead we get the instantiation_error message. Why? Well, when we pose the query this way round, we are asking Prolog to evaluate 12 is (X+3)*2, which it *can't* do as X is not instantiated.

Two final remarks. As we've already mentioned, for Prolog 3 + 2 is just a term. In fact, for Prolog, it really *is* the term *+(3,2)*. The expression 3 + 2 is just a user-friendly notation that's nicer for us to use. This means that if you really want to, you can give Prolog queries like:

X is +(3,2).

and Prolog will correctly reply

X = 5

Actually, you can even given Prolog the query

?- is(X,+(3,2)).

and Prolog will respond

X = 5

This is because, for Prolog, the expression X is +(3,2) is the term is(X,+(3,2)). The expression X is +(3,2) is just user friendly notation. Underneath, as always, Prolog is just working away with terms.

Summing up, arithmetic in Prolog is easy to use. Pretty much all you have to remember is to use "is" to force evaluation, that stuff to be evaluated must goes to the right of "is", and to take care that any variables are correctly instantiated. But there is a deeper lesson that is worth reflecting on. By `bolting on' the extra capability to do arithmetic we have further widened the distance between the procedural and declarative interpretation of Prolog processing.

4.3 Comparing integers

Some Prolog arithmetic predicates actually do carry out arithmetic all by themselves (that is, without the assistance of is). These are the operators that compare integers.

Arithmetic Examples	Prolog Notation
x < y	X < Y.
x <= y	X <= Y.
x = y	X =:= Y.
x = y	X == Y.
$x \neq y$	X =\= Y.
x >= y	X >= Y.
x > y	X > Y.

These operators have the obvious meaning:

?- 2 < 4.
yes

```
?- 2 =< 4.
yes

?- 4 =< 4.
yes

?- 4=:=4.
yes

?- 4=\=5.
yes

?- 4=\=4.
no

?- 4 >= 4.
yes

?- 4 > 2.
yes
```

Moreover, they force both their right-hand and left-hand arguments to be evaluated:

```
?- 2 < 4+1.
yes

?- 2+1 < 4.
yes

?- 2+1 < 3+2.
yes

?- 2 < 1+1+1.
yes

?- 4 > 1+1+1.
yes
```

Note that =:= really is different from =, as the following examples show:

```
?- 4=4.
yes

?- 2+2 =4.
no
```

```
?- 2+2 =:= 4.
yes
```

That is, = tries to unify its arguments; it does not force arithmetic evaluation. That's =:='s job.

Whenever we use these operators, we have to take care that any variables are instantiated. For example, all the following queries lead to instantiation errors.

```
?- X < 3.
 Instantiation Error
?- 3 < Y.
 Instantiation Error
?- X =:= X.
Instantiation Error
?- X < 3,(X=2).
! ----------------------------------------
! Error 22 : Instantiation Error
! Goal     : _39768 is _31480

Aborted
| ?- X =:= X,X=1.
! ----------------------------------------
! Error 22 : Instantiation Error
! Goal     : _30084 is _21798

Aborted
```

Moreover, variables have to be instantiated to integers. By using followed queries:

```
?- X = 3, X < 4.
X = 3 % Succeeds.

?- X=1,X=:=X.
X = 1

?- X=2,X<3.
X = 2
```

But the query

```
?- X = b, X < 4.
Type Error  not
a  function
```

LAB Exercise:

1. Predict the output:

1. ?- write(aaa), write(bbb).
2. ?- write(aaa),nl, write(bbb).
3. ?- writeq(aaa).
4. ?- display(aaa).
5. ?- write('don''t panic').
6. ?- writeq('don''t panic').
7. ?- display('don''t panic').
8. ?- write(Dontpanic).
9. ?- writeq(Dontpanic).
10. ?- display(Dontpanic).
11. ?- write(3.1111*3).
12. ?- display(3.1111*3).
13. ?- write('an\\example').
14. ?- display('an\\example').
15. ?- What is 3.111*3.

2. How does Prolog respond to the following queries using your own understanding?

1. X = 3*4.
2. X is 3*4.
3. 4 is X.
4. X = Y.
5. 3 is 1+2.
6. 3 is +(1,2).
7. 3 is X+2.
8. X is 1+2.
9. 1+2 is 1+2.
10. is(X,+(1,2)).
11. 3+2 = +(3,2).
12. *(7,5) = 7*5.
13. *(7,+(3,2)) = 7*(3+2).
14. *(7,(3+2)) = 7*(3+2).
15. *(7,(3+2)) = 7*(+(3,2)).